

Reinforced Flow Matching

September 18, 2026

Abstract

Large language model with reinforcement learning usually learns a reward-tilted conditional distribution, in a parametric form. In diffusion model, the distribution for the clean data is non-parametric, and represented by samples, and the diffusion model learns a denoising process to approximate the distribution. Inspired by this, we decompose the problem into two steps. reweight the samples according to rewards, and update the flow matching with weighted samples in an incremental way. The reweighting scheme is a non-parametric way for the reward-tilted distribution, formulated from KL-constrained reward maximization. The incremental update consists of the target velocity estimation from the original velocity and velocity correction that depends on mass shift and timestep-dependent contribution weight, followed by an MSE loss for velocity update. We also note that several existing methods can also be casted into sampling reweighting and velocity correction.

1 Incremental velocity target update

The old velocity. Let $p_t(\mathbf{x}_t \mid \mathbf{z})$ be the forward noising distribution and $\mathbf{v}_t(\mathbf{x}_t \mid \mathbf{z})$ its conditional velocity. For example, the linear path $\mathbf{x}_t = (1-t)\mathbf{z} + t\boldsymbol{\epsilon}$, with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, has $\mathbf{v}_t(\mathbf{x}_t \mid \mathbf{z}) = (\mathbf{x}_t - \mathbf{z})/t$ for $t > 0$. The optimal flow-matching velocity is the conditional average

$$\begin{aligned}\mathbf{v}(\mathbf{x}_t, t) &= \mathbb{E}_{\mathbf{z} \sim p(\cdot \mid \mathbf{x}_t)}[\mathbf{v}_t(\mathbf{x}_t \mid \mathbf{z})] \\ &= \mathbb{E}_{\mathbf{z} \sim \pi}[\alpha_t(\mathbf{x}_t, \mathbf{z})\mathbf{v}_t(\mathbf{x}_t \mid \mathbf{z})], \\ \alpha_t(\mathbf{x}_t, \mathbf{z}) &= \frac{p_t(\mathbf{x}_t \mid \mathbf{z})}{p_t(\mathbf{x}_t)}.\end{aligned}\tag{1}$$

Here p_t is the noisy marginal distribution, and α_t measures a sample's relative contribution at $\mathbf{x}_t$. For fixed $(\mathbf{x}_t, t)$, its expectation over $\mathbf{z} \sim \pi$ is one. Reweighting changes the clean distribution while keeping the forward noising process fixed. Hence the new optimal velocity is

$$\mathbf{v}^n(\mathbf{x}_t, t) = \mathbb{E}_{\mathbf{z} \sim \pi^n}[w(\mathbf{z})\alpha_t^n \mathbf{v}_t(\mathbf{x}_t \mid \mathbf{z})].\tag{2}$$

One way: For a small incremental update, we assume that the noisy marginal distribution changes only slightly, so **we assume** $p_t^n(\mathbf{x}_t) \approx p_t^o(\mathbf{x}_t)$, so $\alpha_t^n \approx \alpha_t^o$. Using $\mathbb{E}_{\pi^o}[w - 1] = 0$ gives

$$\begin{aligned} \mathbf{v}^n(\mathbf{x}_t, t) &\approx \mathbb{E}_{z \sim \pi^o}[w(z)\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})] \\ &= \mathbb{E}_{\pi^o}[\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})] + \mathbb{E}_{\pi^o}[(w - 1)\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})] \end{aligned} \quad (3)$$

$$\begin{aligned} &= \mathbf{v}^o(\mathbf{x}_t, t) + \mathbb{E}_{\pi^o}[(w - 1)\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})] \\ &= \mathbf{v}^o(\mathbf{x}_t, t) + \mathbb{E}_{\pi^o}[(w - 1)(\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t))]. \end{aligned} \quad (4)$$

Another way: Since $w = \pi^n / \pi^o$,

$$\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o] = \mathbb{E}_{\mathbf{z} \sim \pi^o}\left[\frac{\pi^n}{\pi^o} \frac{p_t(\mathbf{x}_t | \mathbf{z})}{p_t^o(\mathbf{x}_t)}\right] \quad (5)$$

$$= \mathbb{E}_{\mathbf{z} \sim \pi^n}\left[\frac{p_t(\mathbf{x}_t | \mathbf{z})}{p_t^o(\mathbf{x}_t)}\right] \quad (6)$$

$$= \frac{p_t^n(\mathbf{x}_t)}{p_t^o(\mathbf{x}_t)}, \quad (7)$$

So,

$$\begin{aligned} \mathbf{v}^n(\mathbf{x}_t, t) &= \mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^n \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})] \\ &= \frac{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})]}{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o]}. \end{aligned} \quad (8)$$

The old velocity satisfies $\mathbb{E}_{\mathbf{z} \sim \pi^o}[\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})] = \mathbf{v}^o(\mathbf{x}_t, t)$ and $\mathbb{E}_{\mathbf{z} \sim \pi^o}[\alpha_t^o] = 1$, so

$$\mathbb{E}_{\mathbf{z} \sim \pi^o}[\alpha_t^o(\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t))] = 0. \quad (9)$$

Subtracting this zero term from the numerator gives the exact increment:

$$\begin{aligned} &\mathbf{v}^n(\mathbf{x}_t, t) - \mathbf{v}^o(\mathbf{x}_t, t) \\ &= \frac{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})]}{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o]} - \mathbf{v}^o(\mathbf{x}_t, t) \\ &= \frac{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o(\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t))]}{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o]} \\ &= \frac{\mathbb{E}_{\mathbf{z} \sim \pi^o}[(w - 1)\alpha_t^o(\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t))]}{\mathbb{E}_{\mathbf{z} \sim \pi^o}[w\alpha_t^o]} \\ &= \frac{p_t^o(\mathbf{x}_t)}{p_t^n(\mathbf{x}_t)} \mathbb{E}_{\mathbf{z} \sim \pi^o}[(w - 1)\alpha_t^o(\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t))]. \end{aligned} \quad (10)$$

With the same small-change assumption,

$$\frac{p_t^n(\mathbf{x}_t)}{p_t^o(\mathbf{x}_t)} \approx 1, \quad (11)$$

We have

$$\mathbf{v}^n(\mathbf{x}_t, t) \approx \mathbf{v}^o(\mathbf{x}_t, t) + \mathbb{E}_{\mathbf{z} \sim \pi^o}[(w - 1)\alpha_t^o(\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t))]. \quad (12)$$

This separates three roles: $w - 1$ sets the probability change, α_t^o measures the contribution at the current noisy state, and $\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}^o(\mathbf{x}_t, t)$ gives the correction direction. A weight above one pulls toward that sample’s velocity; a weight below one pushes away. A weight of one contributes no correction.

Estimating the contribution. The marginal density in α_t is expensive to compute. We therefore use a normalized velocity error as a estimate:

$$\begin{aligned} \tilde{\alpha}_t(\mathbf{x}_t, \mathbf{z}) &= \frac{1}{\text{mean} |\mathbf{v}_t(\mathbf{x}_t | \mathbf{z}) - \mathbf{v}_\theta(\mathbf{x}_t, t)|}, \\ \alpha_t(\mathbf{x}_t, \mathbf{z}) &\approx \frac{\tilde{\alpha}_t(\mathbf{x}_t, \mathbf{z})}{\frac{1}{B} \sum_{b=1}^B \tilde{\alpha}_{t_b}(\mathbf{x}_{t_b}, \mathbf{z}_b)}. \end{aligned} \quad (13)$$

The mean in the error is over feature dimensions, B is the batch size. This assigns larger contributions to samples whose conditional velocities are closer to the model prediction.

2 Gap between current implemented algorithm and our derivation

With gradient accumulation over all mini batches before one optimizer update, shuffling samples only changes how the computation is split. The accumulated gradient corresponds to the mean loss over all CK samples. For simplicity, we consider a single prompt and its K samples group below.

Equivalent $\mathbf{v}$ -prediction policy loss of the current implementation.

Let $\mathbf{v}_i = \mathbf{v}_{t_i}(\mathbf{x}_i^{t_i} | \mathbf{x}_i^0)$ be the conditional velocity, and let $\mathbf{v}_i^\theta$ and $\mathbf{v}_i^o$ be the current and old predictions. Using $\mathbf{x}_i^\theta = \mathbf{x}_i^{t_i} - t_i \mathbf{v}_i^\theta$ and the same relation for the old prediction, the algorithm’s policy loss is exactly

$$\mathbf{v}_i^{\text{target}} = \mathbf{v}_i^o + \beta(w_i - 1)(\mathbf{v}_i - \mathbf{v}_i^o), \quad (14)$$

$$\mathcal{L}_{\text{policy}}^{\text{impl}} = \frac{1}{K} \sum_{i=1}^K \frac{t_i \|\mathbf{v}_i^\theta - \mathbf{v}_i^{\text{target}}\|_2^2}{\text{sg}[\text{mean} |\mathbf{v}_i - \mathbf{v}_i^o|]}. \quad (15)$$

Policy loss with the derived target. Using $\mathbf{v}^n$ from Eq. equation 12 as the target and keeping a timestep weight t_i outside MSE, we define

$$\boxed{\begin{aligned} \mathbf{v}_i^{\text{target}} &= \mathbf{v}_i^o + \mathbb{E}_{\mathbf{z} \sim \pi^o}[(w_i - 1)\alpha_{t_i}^o(\mathbf{v}_i - \mathbf{v}_i^o)], \\ \mathcal{L}_{\text{policy}}^{\text{derived}} &= \frac{1}{K} \sum_{i=1}^K t_i \|\mathbf{v}_i^\theta - \mathbf{v}_i^{\text{target}}\|_2^2. \end{aligned}} \quad (16)$$

Algorithm 1 Single Target Loss Base Scheme

Require: Initial model θ_0 , reward function R , prompt distribution $p(c)$, number of prompts per iteration C , rollout size per prompt K , batch size B , total samples per iteration $D = KC$, rollout coverage coefficient β , mass-shift clip Δ_{clip} , reference regularization λ_{ref} , EMA rate η , reward-scale constant ϵ_R , numerical constant ϵ_{alg} , gradient accumulation steps N_{acc} .

Initialize: Training policy $\theta \leftarrow \theta_0$, old policy $\theta_{\text{old}} \leftarrow \theta_0$, reference policy $\theta_{\text{ref}} \leftarrow \theta_0$, data buffer $\mathcal{D} \leftarrow \emptyset$.

- 1: **for** each iteration **do**
- 2: **for** each prompt $c_j \sim p(c)$, $j = 1, \dots, C$ **do** $\triangleright$ Rollout Step, Data Collection
- 3: Sample K clean images $\{\mathbf{x}_0^{j,k}\}_{k=1}^K$ with θ conditioned on c_j .
- 4: Evaluate rewards $R_{j,k} \leftarrow R(\mathbf{x}_0^{j,k}, c_j)$ for $k = 1, \dots, K$.
- 5: **end for**
- 6: Compute prompt-wise reward means $\bar{R}_{c_j} \leftarrow \frac{1}{K} \sum_{k=1}^K R_{j,k}$ for $j = 1, \dots, C$.
- 7: Compute global reward mean $\bar{R} \leftarrow \frac{1}{D} \sum_{j=1}^C \sum_{k=1}^K R_{j,k}$.
- 8: Compute global reward scale $\sigma_R \leftarrow \sqrt{\frac{1}{D} \sum_{j=1}^C \sum_{k=1}^K (R_{j,k} - \bar{R})^2}$.
- 9: **for** each prompt c_j , $j = 1, \dots, C$ **do**
- 10: Compute normalized weights

$$\Delta_{j,k} \leftarrow \frac{1}{\Delta_{\text{clip}}} \text{clip} \left(\frac{R_{j,k} - \bar{R}_{c_j}}{\sigma_R}, -\Delta_{\text{clip}}, \Delta_{\text{clip}} \right), \quad w_{j,k-1} \leftarrow \Delta_{j,k}, \quad k = 1, \dots, K.$$

- 11: Store $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c_j, \mathbf{x}_0^{j,k}, w_{j,k})\}_{k=1}^K$.
- 12: **end for**
- 13: **for** each mini batch $\{(c_i, \mathbf{x}_i^0, w_i)\}_{i=1}^B \subset \mathcal{D}$ **do**
- 14: Sample $t_i \sim \mathcal{U}(0, 1)$, $\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$; set $\mathbf{x}_i^{t_i} \leftarrow (1 - t_i)\mathbf{x}_i^0 + t_i\epsilon_i$.
- 15: Predict $\mathbf{x}_i^\theta$, $\mathbf{x}_i^{\text{old}}$, and $\mathbf{x}_i^{\text{ref}}$ from $(\mathbf{x}_i^{t_i}, c_i, t_i)$ with θ , θ_{old} , and θ_{ref} .
- 16: $\mathbf{x}_i^{\text{target}} \leftarrow \mathbf{x}_i^{\text{old}} + \beta(w_i - 1)(\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}})$.
- 17: Compute

$$\mathcal{L}_{\text{policy}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{\|\mathbf{x}_i^\theta - \mathbf{x}_i^{\text{target}}\|_2^2}{\text{sg}[\text{mean} \|\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}}\|_2]},$$

$$\mathcal{L}_{\text{ref}} \leftarrow \frac{1}{B} \sum_{i=1}^B \|\mathbf{v}_i^\theta - \mathbf{v}_i^{\text{ref}}\|_2^2.$$

- 18: Backpropagate $(\Delta_{\text{clip}} \mathcal{L}_{\text{policy}} + \lambda_{\text{ref}} \mathcal{L}_{\text{ref}}) / N_{\text{acc}}$.
- 19: **end for**
- 20: Update θ ; set $\theta_{\text{old}} \leftarrow \eta \theta_{\text{old}} + (1 - \eta) \theta$ and $\mathcal{D} \leftarrow \emptyset$.
- 21: **end for**

Output: $\mathbf{v}_\theta$

3 Reweighting design

Algorithm 2 Selection scheme: Asymmetric Selection

Require: Initial model θ_0 , reward function R , prompt distribution $p(c)$, C prompts per iteration, K samples per prompt, $D = CK$, batch size B , retained ratio ρ , shift strength β , mass-shift clip Δ_{clip} , reference coefficient λ_{ref} , EMA rate η , accumulation steps N_{acc} .

Initialize: $\theta, \theta_{\text{old}}, \theta_{\text{ref}} \leftarrow \theta_0, \mathcal{D} \leftarrow \emptyset$.

```

1: for each iteration do
2:   for each prompt  $c_j \sim p(c)$ ,  $j = 1, \dots, C$  do
3:     Sample  $\{\mathbf{x}_0^{j,k}\}_{k=1}^K$  with  $\theta$  conditioned on  $c_j$  and set  $R_{j,k} \leftarrow R(\mathbf{x}_0^{j,k}, c_j)$ .
4:   end for
5:   Compute global reward mean  $\bar{R} \leftarrow \frac{1}{D} \sum_{j=1}^C \sum_{k=1}^K R_{j,k}$ .
6:   Compute global reward scale  $\sigma_R \leftarrow \sqrt{\frac{1}{D} \sum_{j=1}^C \sum_{k=1}^K (R_{j,k} - \bar{R})^2}$ .
7:   for each prompt  $c_j \sim p(c)$ ,  $j = 1, \dots, C$  do
8:     Set the mass-shift center to the prompt-group mean:  $b_j \leftarrow K^{-1} \sum_{k=1}^K R_{j,k}$ .
9:     Within this prompt group, compute
           
$$\Delta_{j,k} \leftarrow \frac{1}{\Delta_{\text{clip}}} \text{clip} \left( \frac{R_{j,k} - b_j}{\sigma_R}, -\Delta_{\text{clip}}, \Delta_{\text{clip}} \right)$$

10:    Sort the  $K$  samples so that  $\Delta_{j,1} \leq \dots \leq \Delta_{j,q_j} \leq 0 \leq \Delta_{j,q_j+1} \leq \dots \leq \Delta_{j,K}$ .
11:    Select the positive strong-tail boundary:
           
$$M_j^+ \leftarrow \sum_{k=q_j+1}^K \Delta_{j,k}, \quad h_{j,\rho} \leftarrow \arg \min_{q_j+1 \leq h \leq K} \left| \sum_{k=h}^K \Delta_{j,k} - \rho M_j^+ \right|.$$

12:    Keep all negative samples and the positive strong signal samples:
           
$$s_{j,k} \leftarrow \begin{cases} \Delta_{j,k}, & k \leq q_j, \\ 0, & q_j < k < h_{j,\rho}, \\ \Delta_{j,k}, & k \geq h_{j,\rho}. \end{cases}$$

13:    Normalize the selected mass within this prompt group:
           
$$w_{j,k} - 1 \leftarrow \begin{cases} s_{j,k}, & s_{j,k} < 0, \\ 0, & s_{j,k} = 0, \\ s_{j,k}/\rho, & s_{j,k} > 0. \end{cases}$$

14:    Store  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c_j, \mathbf{x}_0^{j,k}, w_{j,k})\}_{k=1}^K$ .
15:   end for
16:   for each mini batch  $\{(c_i, \mathbf{x}_i^0, w_i)\}_{i=1}^B \subset \mathcal{D}$  do
17:     Sample  $t_i \sim \mathcal{U}(0, 1)$ ,  $\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ; set  $\mathbf{x}_i^{t_i} \leftarrow (1 - t_i)\mathbf{x}_i^0 + t_i\epsilon_i$ .
18:     Predict  $\mathbf{x}_i^\theta$ ,  $\mathbf{x}_i^{\text{old}}$ , and  $\mathbf{x}_i^{\text{ref}}$  from  $(\mathbf{x}_i^{t_i}, c_i, t_i)$  with  $\theta$ ,  $\theta_{\text{old}}$ , and  $\theta_{\text{ref}}$ .
19:      $\mathbf{x}_i^{\text{target}} \leftarrow \mathbf{x}_i^{\text{old}} + \beta(w_i - 1)(\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}})$ .
20:     Compute
           
$$\mathcal{L}_{\text{policy}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{\|\mathbf{x}_i^\theta - \mathbf{x}_i^{\text{target}}\|_2^2}{\text{sg}[\text{mean}[\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}}]]},$$

           
$$\mathcal{L}_{\text{ref}} \leftarrow \frac{1}{B} \sum_{i=1}^B \|\mathbf{v}_i^\theta - \mathbf{v}_i^{\text{ref}}\|_2^2.$$

21:   Backpropagate  $(\Delta_{\text{clip}}\mathcal{L}_{\text{policy}} + \lambda_{\text{ref}}\mathcal{L}_{\text{ref}})/N_{\text{acc}}$ .
22:   end for
23:   Update  $\theta$ ; set  $\theta_{\text{old}} \leftarrow \eta\theta_{\text{old}} + (1 - \eta)\theta$  and  $\mathcal{D} \leftarrow \emptyset$ .
24: end for
Output:  $\mathbf{x}_\theta$ 

```

We now use Algorithm 2 to study how raw rewards should determine the probability ratio w . For one prompt, let $R_1, \dots, R_K$ be the rewards of its K samples. Consider the group-wise empirical distribution, the old probability of each sample is $1/K$. A nonnegative weight with group mean equal to one defines the

new probability w_i/K for each sample, so

$$w_i = \frac{\pi^n(\mathbf{z}_i)}{\pi^o(\mathbf{z}_i)}, \quad w_i \geq 0, \quad \frac{1}{K} \sum_{i=1}^K w_i = 1. \quad (17)$$

The current target uses $\beta(w_i - 1)$ in Eq. equation 14, and β control the step size of target shift.

3.1 Concentrate the confident positive updates

Base signal. Let $\bar{R} = K^{-1} \sum_{i=1}^K R_i$:

$$\Delta_i = \frac{1}{\Delta_{\text{clip}}} \text{clip}\left(\frac{R_i - \bar{R}}{\sigma_R}, -\Delta_{\text{clip}}, \Delta_{\text{clip}}\right). \quad (18)$$

Here $\bar{R}$ is the prompt-group mean, while σ_R is the reward standard deviation over the full rollout samples, as in the DiffusionNFT implementation. The base rule is $w_i - 1 = \Delta_i$. Positive signals pull toward above-average samples and negative signals push away from below-average samples linearly.

Selection scheme: Keep all negative signals, select strong positive signals. The idea is to move positive update strength from slightly above-average samples to better samples. Sort the positive Δ_i from largest to smallest and retain a fraction $\rho \in (0, 1]$ of their total signal mass. Let s_i be the retained positive signal. We keep samples up to the boundary closest to the desired mass. Stronger positive samples have $s_i = \Delta_i$; weaker ones have $s_i = 0$. So that

$$\sum_{i:\Delta_i>0} s_i = \rho \sum_{i:\Delta_i>0} \Delta_i. \quad (19)$$

Finally sets

$$w_i - 1 = \begin{cases} \Delta_i, & \Delta_i \leq 0, \\ s_i/\rho, & \Delta_i > 0. \end{cases} \quad (20)$$

Dividing by ρ restores the total positive signal after selection. The negative side is unchanged. When $\rho = 1$, this reduces to the base scheme. An unselected positive sample has $w_i = 1$: its target stays at the old prediction. This scheme tests whether positive updates benefit from more selective allocation.

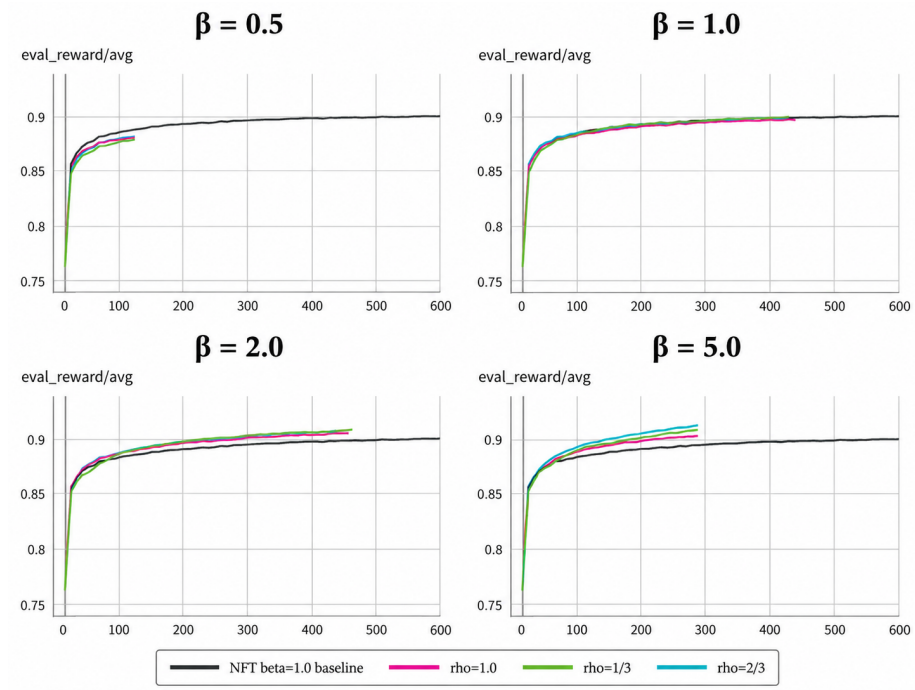


Figure 1: Ablation for selection scheme on Pickscore.

Evidence at larger β .

3.2 From simple selection to reward mappings

Algorithm 3 Exploration of reward mappings to probability ratio w

Require: Initial model θ_0 , reward function R , prompt distribution $p(c)$, number of prompts per iteration C , rollout size per prompt K , batch size B , total samples per iteration $D = KC$, rollout coverage coefficient β , mass-shift clip Δ_{clip} , reference regularization λ_{ref} , EMA rate η , reward-scale constant ϵ_R , numerical constant ϵ_{alg} , gradient accumulation steps N_{acc} .

Initialize: Training policy $\theta \leftarrow \theta_0$, old policy $\theta_{\text{old}} \leftarrow \theta_0$, reference policy $\theta_{\text{ref}} \leftarrow \theta_0$, data buffer $\mathcal{D} \leftarrow \emptyset$.

```

1: for each iteration do
2:   for each prompt  $c_j \sim p(c)$ ,  $j = 1, \dots, C$  do           ▷ Rollout Step, Data
      Collection
3:     Sample  $K$  clean images  $\{\mathbf{x}_0^{j,k}\}_{k=1}^K$  with  $\theta$  conditioned on  $c_j$ .
4:     Evaluate rewards  $R_{j,k} \leftarrow R(\mathbf{x}_0^{j,k}, c_j)$  for  $k = 1, \dots, K$ .
5:   end for
6:   for each prompt  $c_j$ ,  $j = 1, \dots, C$  do
7:     Compute normalized weights

```

$$w_{j,k} \leftarrow \frac{f(R_{j,k})}{\frac{1}{K} \sum_{k=1}^K f(R_{j,k})}, \quad k = 1, \dots, K.$$

```

8:     Store  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c_j, \mathbf{x}_0^{j,k}, w_{j,k})\}_{k=1}^K$ .
9:   end for
10:  for each mini batch  $\{(c_i, \mathbf{x}_i^0, w_i)\}_{i=1}^B \subset \mathcal{D}$  do
11:    Sample  $t_i \sim \mathcal{U}(0, 1)$ ,  $\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ; set  $\mathbf{x}_i^{t_i} \leftarrow (1 - t_i)\mathbf{x}_i^0 + t_i\epsilon_i$ .
12:    Predict  $\mathbf{x}_i^\theta$ ,  $\mathbf{x}_i^{\text{old}}$ , and  $\mathbf{x}_i^{\text{ref}}$  from  $(\mathbf{x}_i^{t_i}, c_i, t_i)$  with  $\theta$ ,  $\theta_{\text{old}}$ , and  $\theta_{\text{ref}}$ .
13:     $\mathbf{x}_i^{\text{target}} \leftarrow \mathbf{x}_i^{\text{old}} + \beta(w_i - 1)(\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}})$ .
14:    Compute

```

$$\mathcal{L}_{\text{policy}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{\|\mathbf{x}_i^\theta - \mathbf{x}_i^{\text{target}}\|_2^2}{\text{sg}[\text{mean}[\|\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}}\|]]},$$

$$\mathcal{L}_{\text{ref}} \leftarrow \frac{1}{B} \sum_{i=1}^B \|\mathbf{v}_i^\theta - \mathbf{v}_i^{\text{ref}}\|_2^2.$$

```

15:    Backpropagate  $(\Delta_{\text{clip}}\mathcal{L}_{\text{policy}} + \lambda_{\text{ref}}\mathcal{L}_{\text{ref}})/N_{\text{acc}}$ .
16:  end for
17:  Update  $\theta$ ; set  $\theta_{\text{old}} \leftarrow \eta\theta_{\text{old}} + (1 - \eta)\theta$  and  $\mathcal{D} \leftarrow \emptyset$ .
18: end for

```

Output: $\mathbf{v}_\theta$

We extend the selection idea by using a reward map f to define the new probability of each sample. For a fixed prompt c_j , consider its K samples with

rewards $R_{j,1}, \dots, R_{j,K}$. As in Eq. equation 17, we work with the empirical group distribution and omit the prompt conditioning from π :

$$\pi^o(\mathbf{z}_i) = \frac{1}{K}, \quad \pi^n(\mathbf{z}_i) = \frac{f(R_{j,i})}{\sum_{\ell=1}^K f(R_{j,\ell})}. \quad (21)$$

The corresponding probability ratio is therefore

$$w_{j,i} = \frac{\pi^n(\mathbf{z}_i)}{\pi^o(\mathbf{z}_i)} = \frac{f(R_{j,i})}{\frac{1}{K} \sum_{\ell=1}^K f(R_{j,\ell})}. \quad (22)$$

For nonnegative f with a positive sum, this construction gives $w_{j,i} \geq 0$ and $K^{-1} \sum_i w_{j,i} = 1$. Thus, $w_{j,i}$ is a probability ratio, while $w_{j,i}/K$ is the new sample probability. The signed change $w_{j,i} - 1$ controls the target correction: $w_{j,i} > 1$ increases the sample’s probability in the desired distribution, $w_{j,i} < 1$ decreases it, and $w_{j,i} = 1$ leaves it unchanged. If all mapped rewards are zero, we keep the old uniform distribution by setting $w_{j,i} = 1$ for all samples. For mappings that depend on the reward statistics of each prompt group, we define

$$\bar{R}_{c_j} = \frac{1}{K} \sum_{i=1}^K R_{j,i}, \quad \sigma_{c_j} = \sqrt{\frac{1}{K} \sum_{i=1}^K (R_{j,i} - \bar{R}_{c_j})^2}.$$

Unless stated otherwise, the experiments below normalize mapped rewards within each prompt using Eq. equation 22.

Motivation and experiment plan. **Experiment A** pushes this idea to the extreme situation. A one-hot rule gives all weight to the top sample(s) and zero to the rest. This sharp choice may make training unstable, so we also give weight to samples within $cR_j^{\max}$ of the top reward, without using the group standard deviation for this threshold. If no reward in the group is positive, we use uniform weights and apply no reward-driven target shift to that group.

Experiment B straightly tests the idea with softmax. Compared with $c = 1$, $c < 1$ puts more weight on high-reward samples, while $c > 1$ spreads weight more evenly. If $c < 1$ works better, it supports our idea.

Experiment C uses the unshifted linear mapping as a baseline. Experiment D compares sigmoid with linear within each group: depending on c and the rewards, sigmoid can yield either sharper or flatter normalized weights. We test whether smaller c helps, whether $c = 0.3$ underperforms, and whether a small- c sigmoid is preferable to the softmax-style mappings.

Experiment E changes the power p to test whether steeper or flatter maps work better. It is another view to test our idea.

Experiment F uses an L1 scale. Its weight curve looks much like the softmax curve in Exp.B, so it is less important and may be skipped.

Experiment A: One-hot and near-maximum threshold. Let

$$R_j^{\max} = \max_i R_{j,i}, \quad T_j(c) = R_j^{\max} - cR_j^{\max} = (1 - c)R_j^{\max}.$$

$$S_j(c) = \{i : R_{j,i} \geq T_j(c)\} \quad \text{for } R_j^{\max} > 0.$$

Set

$$w_{j,i}(c) = \begin{cases} \frac{K}{|S_j(c)|} \mathbb{1}[i \in S_j(c)], & R_j^{\max} > 0, \\ 1, & R_j^{\max} \leq 0, \end{cases} \quad c \in \{0, 0.05, 0.1, 0.25\}.$$

For $R_j^{\max} > 0$, when the maximum is unique, $c = 0$ is one-hot with weight K on the max-reward sample. If several samples are equal-reward at the maximum, they share the K mass equally. Increasing c includes samples within a larger fraction of the maximum reward.

For $R_j^{\max} \leq 0$, all $w_{j,i}(c)$ equal 1, so the probability for each sample in this group c_j will not change. While other loss terms like mse between v_θ and v_{old} and KL terms still apply.

Target: Test whether assigning most or all of each group’s weight mass to its best-reward samples is the main source of the gain from softmax weighting.

Experiment B: Softmax with standard-deviation temperature.

$$f_c(R_{j,i}) = \exp\left(\frac{R_{j,i}}{c\sigma_{c_j}}\right), \quad c \in \{0.5, 0.75, 1.0, 1.5\}.$$

For $\sigma_{c_j} > 0$, smaller c yields a steeper mapping and more concentrated weights; larger c gives a flatter mapping.

Target: This exp tests the trade-off between concentrating on high rewards and keeping a flatter training signal. We want to test whether more concentrating or flat than classical softmax is better for optimize.

Experiment C: Linear baseline.

$$f_{\text{lin}}(R) = [R]_+, \quad f_{\text{lin,shift}}(R) = [1 + R]_+, \quad [x]_+ = \max(x, 0).$$

Only $f_{\text{lin}}(R) = [R]_+$ is included in the planned sweep; the shifted mapping is shown for comparison, not run as an experiment.

Target: Linear reward mapping acts as baseline variant. Expect to be worse than softmax and one-hot.

Experiment D: Sigmoid mapping.

$$f_{\text{sig},c}(R_{j,i}) = \frac{1}{1 + \exp(-(R_{j,i} - 0.5)/c)}, \quad c \in \{0.01, 0.1, 0.3\}.$$

The smaller c is, the sharper the transition around $R = 0.5$. After within-group normalization, the sigmoid may be sharper or flatter than linear, depending on the reward range; its saturation also limits how much the highest rewards can dominate.

Target: Compare sigmoid with the linear baseline within each group. Test whether smaller c improves performance, whether the flatter $c = 0.3$ setting underperforms, and whether a small- c sigmoid is a better choice than the softmax-style mappings in Experiment B.

Experiment E: Power mapping.

$$f_p(R_{j,i}) = [R_{j,i}]_+^p.$$

We consider $p \in \{0.5, 2.0\}$.

Target: Compare with the linear $p = 1$ to test whether the stronger emphasis on high rewards gives better behavior (similar performance with softmax).

Experiment F: Softmax with L1 scale. Define

$$\|R_{j,i} - \bar{R}_{c_j}\|_1 = \frac{1}{K} \sum_{i=1}^K |R_{j,i} - \bar{R}_{c_j}|.$$

Then

$$f_{c,1}(R_{j,i}) = \exp\left(\frac{R_{j,i}}{c\|R_{j,i} - \bar{R}_{c_j}\|_1}\right), \quad c \in \{0.5, 0.75, 1.0, 1.5\}.$$

Target: This is a variant for softmax. It is similar with softmax with std temperature as shown in following figure. So we expect a similar performance with Experiment B. This exp can be skipped if necessary.

Experiment G: Exponential on positive excess with mass-preserving rescaling. As in Experiment C, we use the linear mapping

$$w_{j,i} = \begin{cases} \frac{[R_{j,i}]_+}{\frac{1}{K} \sum_{k=1}^K [R_{j,k}]_+}, & \sum_{k=1}^K [R_{j,k}]_+ > 0, \\ 1, & \sum_{k=1}^K [R_{j,k}]_+ = 0, \end{cases}$$

so that $\frac{1}{K} \sum_{i=1}^K w_{j,i} = 1$. We apply an exponential only to the positive excess, while leaving the non-positive excess unchanged:

$$h_{j,i}(\alpha) = \begin{cases} \exp(\alpha(w_{j,i} - 1)), & w_{j,i} - 1 > 0, \\ w_{j,i} - 1, & w_{j,i} - 1 \leq 0, \end{cases} \quad \alpha \in \{1.0, 5.0, 10.0, 20.0\}.$$

Because the exponential changes the total positive mass, we rescale the positive part so that its sum is preserved:

$$S_j^+ = \sum_{i: w_{j,i}-1>0} (w_{j,i} - 1), \quad \tilde{S}_j^+(\alpha) = \sum_{i: w_{j,i}-1>0} h_{j,i}(\alpha),$$

$$\tilde{w}_{j,i} = \begin{cases} \frac{S_j^+}{\tilde{S}_j^+(\alpha)} h_{j,i}(\alpha), & w_{j,i} - 1 > 0, \\ w_{j,i} - 1, & w_{j,i} - 1 \leq 0. \end{cases}$$

Since $\sum_i: w_{j,i-1} > 0 \tilde{w}_{j,i} = S_j^+$ and the negative part is untouched, we still have $\sum_{i=1}^K \tilde{w}_{j,i} = 0$. The target shift then uses the same form as the algorithm:

$$\mathbf{x}_i^{\text{target}} = \mathbf{x}_i^{\text{old}} + \beta \tilde{w}_{j,i} (\mathbf{x}_i^0 - \mathbf{x}_i^{\text{old}}).$$

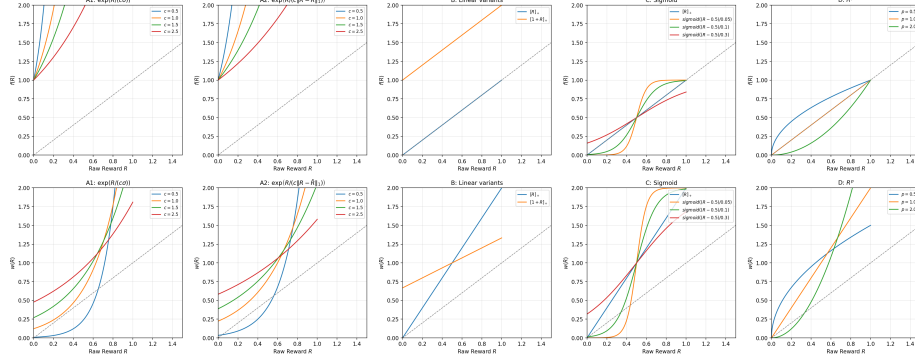


Figure 2: Illustration of the temperature multiplier c with $\bar{R}_{c_j} = 0.3$, $\sigma_{c_j} = 0.5$, and mean centered L1 deviation $d_{1,c_j} = 0.4$.

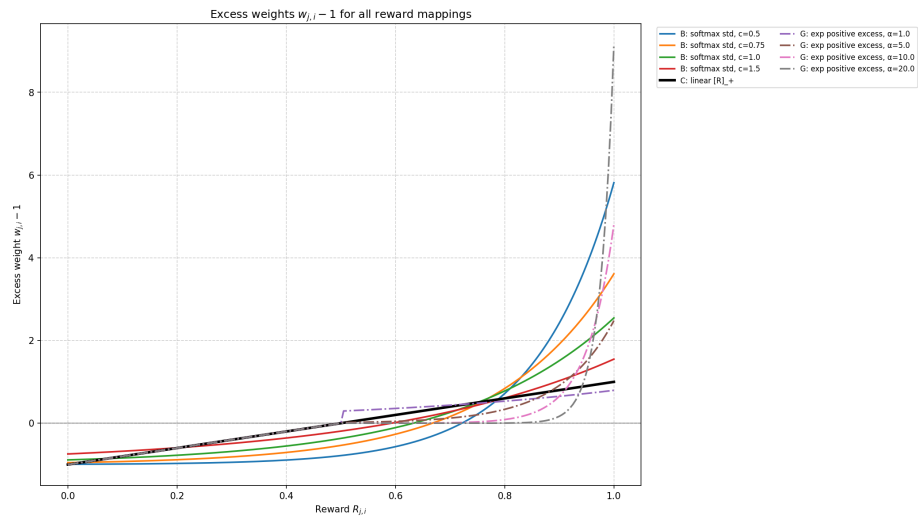


Figure 3: Comparison Chart of Curves from Experiment G